

BÁO CÁO KẾT QUẢ THỬ NGHIỆM

Thời gian thực hiện: 01/10 – 10/10/2026

Sinh viên thực hiện:

Họ Tên: Võ Thị Hồng Gấm

MSSV: 25520456

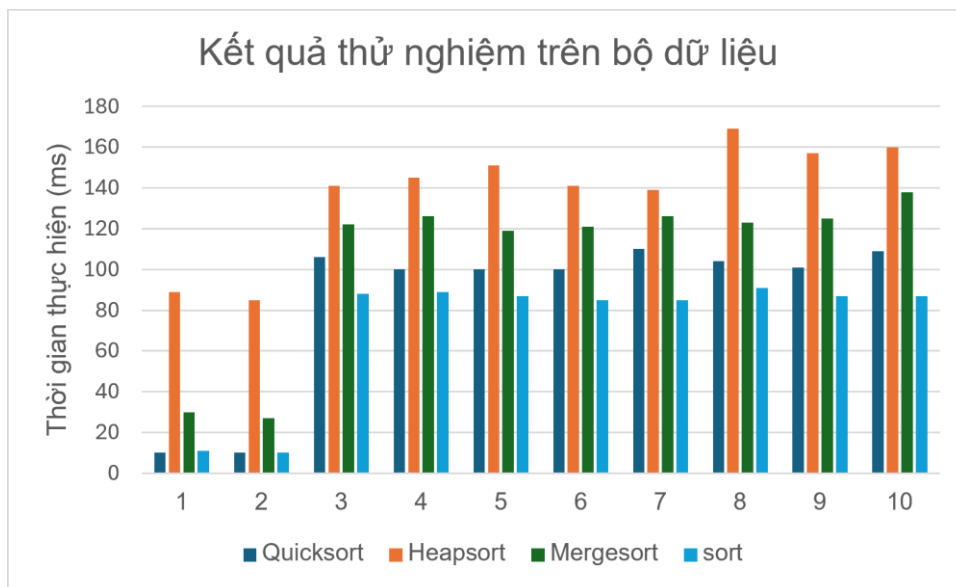
Nội dung báo cáo:

I. Kết quả thử nghiệm

1. Bảng thời gian thực hiện

Dữ liệu	Thời gian thực hiện (ms)			
	Quicksort	Heapsort	Mergesort	sort (C++)
1	10	89	30	11
2	10	85	27	10
3	106	141	122	88
4	100	145	126	89
5	100	151	119	87
6	100	141	121	85
7	110	139	126	85
8	104	169	123	91
9	101	157	125	87
10	109	160	138	87
Trung bình	85	137.7	105.7	72

2. Biểu đồ (cột) thời gian thực hiện



II. Kết luận:

- Qua thử nghiệm trên 10 dãy dữ liệu (mỗi dãy 1 triệu số thực), hàm sort của C++ cho thời gian trung bình tốt nhất (72 ms), tiếp theo là QuickSort (85 ms), MergeSort (105.7 ms) và chậm nhất là HeapSort (137.7 ms). Hàm sort nhanh nhất

vì là hàm thư viện đã được tối ưu sẵn, còn trong ba thuật toán tự cài đặt thì QuickSort cho kết quả tốt nhất.

- Với 2 dãy đã sắp xếp sẵn tăng dần và giảm dần (data1, data2), QuickSort và sort chỉ mất khoảng 10ms, MergeSort mất khoảng 27 đến 30 ms, trong khi HeapSort vẫn mất khoảng 85 đến 89 ms. Vì HeapSort không tận dụng được dữ liệu đã sắp xếp, nó luôn xếp lại thành mảng rồi lấy từng phần tử, nên HeapSort gần như không được lợi gì từ dữ liệu đã có thứ tự.

- Với 8 dãy ngẫu nhiên còn lại (từ data3 đến data10), thời gian của các thuật toán đều tăng lên rõ rệt và khá ổn định giữa các dãy. HeapSort chậm nhất vì phải so sánh và đổi chỗ các phần tử nằm xa nhau trong bộ nhớ, còn MergeSort chậm hơn QuickSort vài lần cần thêm mảng phụ để trộn và sao chép dữ liệu qua lại. Cả ba thuật toán đều có độ phức tạp trung bình $O(n \log n)$ nên thời gian chạy cùng cấp độ, sự chênh lệch chủ yếu đến từ cách cài đặt và cách truy cập bộ nhớ.

III. Thông tin chi tiết – [link github](#), trong report gibub cần có

1. Báo cáo: [link](#)
2. Mã nguồn: [link](#)
3. Dữ liệu thử nghiệm: [link](#)